LABORATÓRIO: PROGRAMAÇÃO DE MEMÓRIA COMPARTILHADA COM OPENMP

Exercícios 1 e 2

Faculdade de Computação e Informática – FCI
Aluno: Pedro de Souza Zequi
RA: 10419805

---

1. Introdução

Este relatório apresenta a resolução dos Exercícios 1 e 2 do laboratório "Explorando OpenMP". No Exercício 1, a integral pela regra do trapézio foi implementada em quatro versões, seguindo a evolução dos slides 24 a 38 do material de aula. No Exercício 2, foi feito o somatório de um vetor de 2^{30} números em duas versões paralelas, com a avaliação do desempenho (escalabilidade forte e fraca). Todos os programas foram escritos em C, compilados com gcc e a flag -fopenmp, e executados no GitHub Codespaces.

2. Exercício 1 – Integral pela Regra do Trapézio

Foi calculada a integral de `f(x) = x²` no intervalo [0, 3] com n = 1024 trapézios. O valor exato é 9. Cada versão foi executada com 1, 2 e 4 threads, passando os valores de a, b e n pela entrada padrão (`echo "0 3 1024" | ./trapX <threads>`).

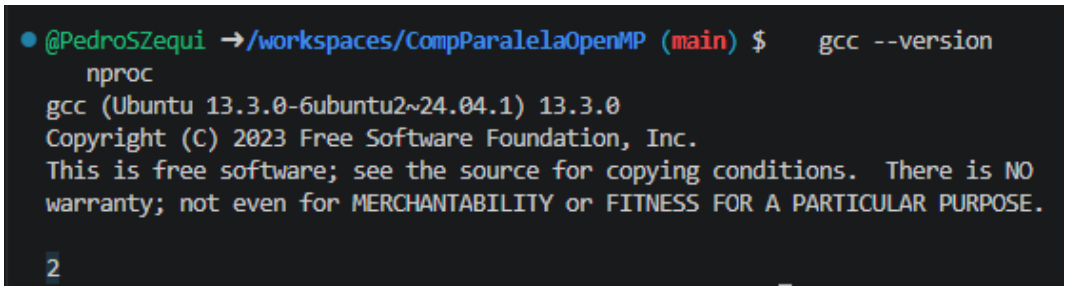


Figura 1 – Ambiente utilizado no Exercício 1 (versão do gcc e número de processadores)

2.1 Versão 1 – critical dentro da função Trap (slides 24–25)

Cada thread calcula a sua fatia do intervalo dentro de `Trap()` e soma o resultado parcial na variável global por meio de um ponteiro, protegida por `#pragma omp critical`. O número de trapézios precisa ser divisível pelo número de threads.

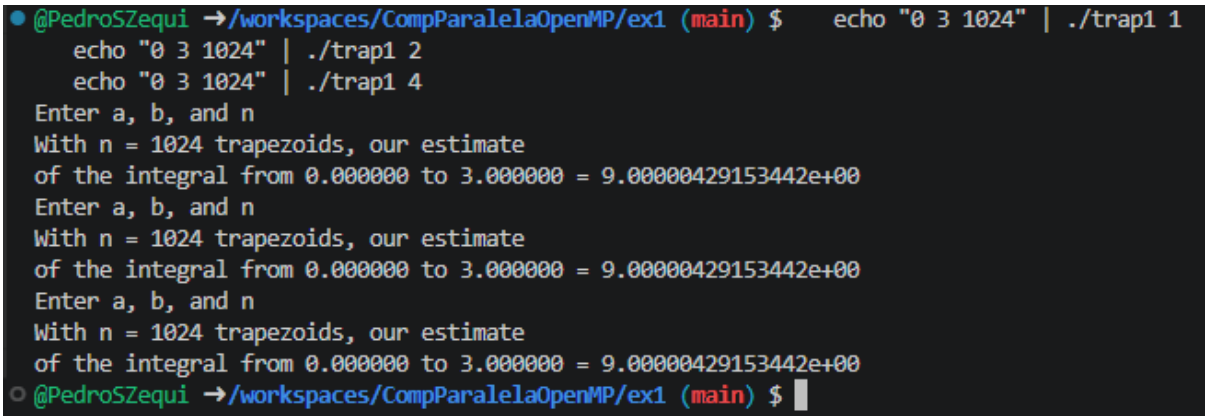


Figura 2 – Execução da Versão 1 com 1, 2 e 4 threads

2.2 Versão 2 – variável privada e critical fora da função (slide 32)

A função passa a se chamar `Local_trap()` e retorna o resultado. Dentro do bloco paralelo, cada thread guarda esse valor numa variável privada `my_result`, e só a soma final fica dentro da seção crítica. Assim, o cálculo pesado não é serializado.

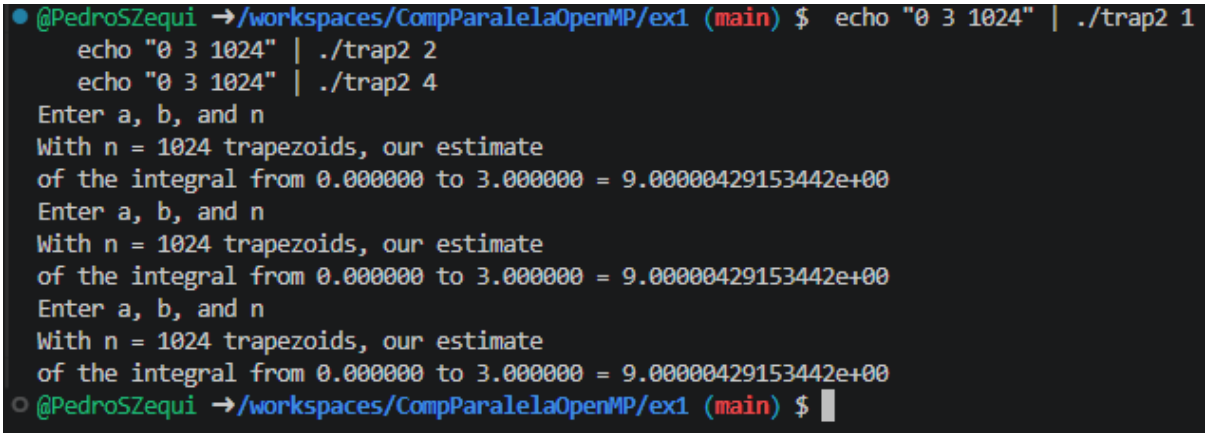


Figura 3 – Execução da Versão 2 com 1, 2 e 4 threads

2.3 Versão 3 – cláusula reduction (slide 35)

A seção crítica é substituída pela cláusula `reduction(+: global_result)` na diretiva `parallel`. O OpenMP cria uma cópia privada da variável para cada thread e soma todas as cópias ao final da região paralela.

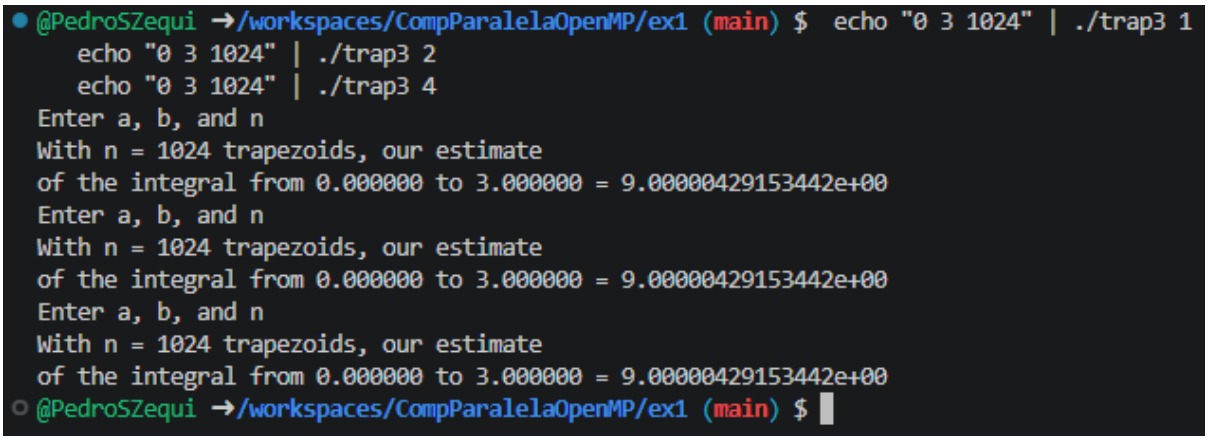


Figura 4 – Execução da Versão 3 com 1, 2 e 4 threads

2.4 Versão 4 – parallel for com reduction (slide 38)

O código volta a ser praticamente igual ao serial: basta a diretiva `#pragma omp parallel for reduction(+: approx)` sobre o laço. O OpenMP divide as iterações entre as threads automaticamente, e por isso n não precisa mais ser divisível pelo número de threads. A última execução (n = 1000 com 3 threads) demonstra isso.

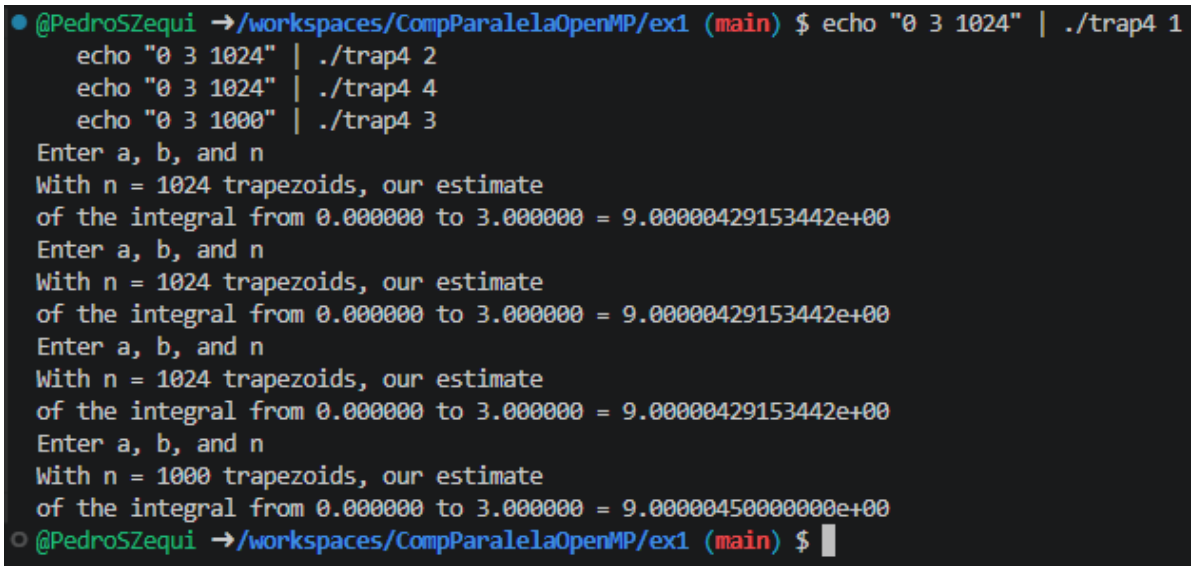


Figura 5 – Execução da Versão 4 com 1, 2, 4 e 3 threads (esta última com n = 1000)

2.5 Análise dos resultados

As quatro versões produziram o mesmo resultado, 9,00000429153442, para qualquer número de threads. A diferença em relação ao valor exato ($\approx 4{,}3 \times 10^{-6}$) não é um erro do programa: é o erro de aproximação da própria regra do trapézio, que para $f(x) = x^2$ vale $(b - a)\cdot h^2 \cdot f''/12 \approx 4{,}29 \times 10^{-6}$. Com n = 1000, o erro é um pouco maior ($4{,}5 \times 10^{-6}$), porque os trapézios são mais largos.

3. Exercício 2 – Avaliação de Desempenho (Somatório de Vetor)

3.1 Implementação

O vetor é preenchido com `v[i] = (i % 100) + 1`, o que garante que os valores, e portanto a soma final, sejam sempre os mesmos. Foram construídas duas versões paralelas:

- **Versão A (critical):** cada thread soma a sua parte do vetor numa variável privada, com as iterações divididas por `#pragma omp for`, e depois acumula o resultado na variável compartilhada dentro de `#pragma omp critical`.
- **Versão B (reduction):** `#pragma omp parallel for reduction(+: total)`. A soma das parciais é feita automaticamente pelo OpenMP.

O próprio programa executa cada configuração (1 a 6 threads) 10 vezes, mede apenas o tempo da soma com `omp_get_wtime()`, calcula a média e confere o resultado com uma soma serial de referência. Antes das medições é feita uma execução de aquecimento, que não entra na média. Os programas foram compilados com `gcc -O2 -Wall -fopenmp`.

- **Escalabilidade forte:** N fixo = 2^{30} (1.073.741.824 elementos, 4 GB de dados em memória). Speedup $S = T_1 / T_p$.
- **Escalabilidade fraca:** cada thread processa ≈ $2^{30}/6$ elementos, ou seja, N = p × 178.956.970, chegando a ≈ 2^{30} com 6 threads. Speedup escalado $S = p \times T_1 / T_p$.

3.2 Ambiente de execução

As medições foram feitas num Codespace com processador AMD EPYC 7763, 2 núcleos físicos com SMT (4 threads de hardware) e 16 GB de RAM.

```
@PedroSZequi →/workspaces/CompParalelaOpenMP (main) $ cd /workspaces/CompParalelaOpenMP
nproc
lscpu | grep -E "Model name|Socket|Core|Thread"
free -h
4
Model name:                           AMD EPYC 7763 64-Core Processor
Thread(s) per core:                   2
Core(s) per socket:                   2
Socket(s):                            1
               total        used        free      shared  buff/cache   available
Mem:            15Gi       2.1Gi       3.1Gi        61Mi        10Gi        13Gi
Swap:             0B          0B          0B
@PedroSZequi →/workspaces/CompParalelaOpenMP (main) $
```

Figura 6 – Ambiente de execução do Exercício 2 (nproc, lscpu e memória)

3.3 Execuções

```
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex2 (main) $ ./soma_critical forte 30 | tee saida_critical_forte.txt
Versao: critical | Escalabilidade: forte | 2^30 | 10 repeticoes | nproc=4
threads  N           media(s)     speedup     soma           confere?
1        1073741824  0.379511     1.000       54223961200    OK
2        1073741824  0.193758     1.959       54223961200    OK
3        1073741824  0.209914     1.808       54223961200    OK
4        1073741824  0.205835     1.844       54223961200    OK
5        1073741824  0.221995     1.710       54223961200    OK
6        1073741824  0.210077     1.807       54223961200    OK
CSV gravado em resultado_critical_forte.csv
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex2 (main) $
```

Figura 7 – Versão critical – escalabilidade forte ($N = 2^{30}$)

```
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex2 (main) $ ./soma_reduction forte 30 | tee saida_reduction_forte.txt
Versao: reduction | Escalabilidade: forte | 2^30 | 10 repeticoes | nproc=4
threads  N           media(s)     speedup     soma           confere?
1        1073741824  0.392115     1.000       54223961200    OK
2        1073741824  0.195158     2.009       54223961200    OK
3        1073741824  0.210243     1.865       54223961200    OK
4        1073741824  0.200419     1.956       54223961200    OK
5        1073741824  0.204693     1.916       54223961200    OK
6        1073741824  0.197696     1.983       54223961200    OK
CSV gravado em resultado_reduction_forte.csv
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex2 (main) $
```

Figura 8 – Versão reduction – escalabilidade forte ($N = 2^{30}$)

```
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex2 (main) $ ./soma_critical fraca 30 | tee saida_critical_fraca.txt
Versao: critical | Escalabilidade: fraca | 2^30 | 10 repeticoes | nproc=4
threads  N           media(s)     speedup     soma           confere?
1        178956970   0.061788     1.000       9037325935     OK
2        357913940   0.066184     1.867       18074652770    OK
3        536870910   0.116517     1.591       27111980505    OK
4        715827880   0.146367     1.689       36149307140    OK
5        894784850   0.180598     1.711       45186633675    OK
6        1073741820  0.217841     1.702       54223961110    OK
CSV gravado em resultado_critical_fraca.csv
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex2 (main) $
```

Figura 9 – Versão critical – escalabilidade fraca (até 2^{30})

```
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex2 (main) $ ./soma_reduction fraca 30 | tee saida_reduction_fraca.txt
Versao: reduction | Escalabilidade: fraca | 2^30 | 10 repeticoes | nproc=4
threads  N           media(s)     speedup     soma           confere?
1        178956970   0.062929     1.000       9037325935     OK
2        357913940   0.064238     1.959       18074652770    OK
3        536870910   0.120891     1.562       27111980505    OK
4        715827880   0.135510     1.858       36149307140    OK
5        894784850   0.175333     1.795       45186633675    OK
6        1073741820  0.203789     1.853       54223961110    OK
CSV gravado em resultado_reduction_fraca.csv
@PedroSZequi ➜/workspaces/CompParalelaOpenMP/ex2 (main) $
```

Figura 10 – Versão reduction – escalabilidade fraca (até 2^{30})

3.4 Resumo dos resultados

As tabelas abaixo resumem os tempos médios das 10 execuções e o speedup. As tabelas completas, com todos os tempos individuais, e os gráficos de speedup estão na planilha Excel entregue, que tem uma aba para cada versão.

Tabela 1 – Escalabilidade forte ($N = 2^{30}$) – tempo médio (s) e speedup

Threads	Critical – tempo	Critical – speedup	Reduction – tempo	Reduction – speedup
1	0,3795	1,00	0,3921	1,00
2	0,1938	1,96	0,1952	2,01
3	0,2099	1,81	0,2102	1,86
4	0,2058	1,84	0,2004	1,96
5	0,2220	1,71	0,2047	1,92
6	0,2101	1,81	0,1977	1,98

Tabela 2 – Escalabilidade fraca – tempo médio (s) e speedup escalado

Threads	N	Critical – tempo	Critical – speedup	Reduction – tempo	Reduction – speedup
1	178.956.970	0,0618	1,00	0,0629	1,00
2	357.913.940	0,0662	1,87	0,0642	1,96
3	536.870.910	0,1165	1,59	0,1209	1,56
4	715.827.880	0,1464	1,69	0,1355	1,86
5	894.784.850	0,1806	1,71	0,1753	1,79
6	1.073.741.820	0,2178	1,70	0,2038	1,85

4. Conclusão

No Exercício 1, as quatro versões chegaram ao mesmo resultado, o que mostra que todas estão corretas. A evolução entre elas mostra como o OpenMP simplifica a programação paralela: o código começou com a divisão manual do trabalho e uma seção crítica explícita e terminou com uma única diretiva parallel for com reduction, que é mais curta, mais legível e não exige que n seja divisível pelo número de threads.

No Exercício 2, as duas versões calcularam a soma corretamente em todas as execuções. Com 2 threads, o speedup chegou a cerca de 2, próximo do ideal para os 2 núcleos físicos da máquina. A partir de 3 threads não houve ganho adicional: as threads extras dividem os mesmos núcleos físicos (SMT) e, principalmente, a mesma banda de memória. Como a soma faz uma única operação por elemento lido, o programa é limitado pela velocidade da memória, e não pela CPU. Na escalabilidade fraca, de 1 para 2 threads o tempo ficou quase constante com o dobro de dados, e depois passou a crescer junto com N.

As versões critical e reduction tiveram desempenho praticamente igual, pois o critical é executado só uma vez por thread. A versão com reduction, porém, é mais simples e menos sujeita a erros.